

Rodrigo Fernandez

Senior DevOps Engineer

Katy, Texas | +1 (407) 801-1287 | <https://www.linkedin.com/in/rodrigo-bermejo-fern%C3%A1ndez-9a5a0664/> | rodrigobfdev@gmail.com

Summary

DevOps Engineer with 10+ years of experience supporting **FinTech**, **healthcare**, **SaaS**, **eCommerce**, and enterprise infrastructure platforms, specializing in **AWS**, **Azure**, **GCP**, **Kubernetes 1.18–1.29**, **Docker 19.x–26.x**, **Terraform 0.12–1.8**, **Jenkins 2.x**, **GitHub Actions**, **GitLab CI 12.x–17.x**, **Ansible 2.8–2.16**, **Helm 3.x**, **Argo CD 1.x–2.x**, **Prometheus 2.x**, and **Grafana 7.x–10.x** to modernize CI/CD, automate cloud provisioning, migrate legacy deployment flows, improve observability, resolve production issues, reduce release risk, and build secure, repeatable DevOps platforms for fast-moving product teams.

Skills

- **Cloud Platforms:** **AWS**, Azure, GCP, EC2, EKS, ECS, Lambda, RDS, S3, CloudFront, Route 53, IAM, VPC, ALB, CloudWatch, Azure DevOps, GKE, Cloud Run, Secret Manager
- **Infrastructure as Code:** **Terraform 0.12–1.8**, CloudFormation, Ansible 2.8–2.16, Terragrunt, reusable modules, remote state, policy-based provisioning
- **Containers & Orchestration:** **Docker 19.x–26.x**, **Kubernetes 1.18–1.29**, Helm 3.x, EKS, GKE, Ingress, HPA, Cluster Autoscaler, service mesh basics
- **CI/CD & Release Engineering:** **GitHub Actions**, **GitLab CI 12.x–17.x**, **Jenkins 2.x**, Argo CD 1.x–2.x, blue-green deployment, canary release, rollback automation
- **Monitoring & Observability:** **Prometheus 2.x**, **Grafana 7.x–10.x**, CloudWatch, ELK, Loki, OpenTelemetry, alert routing, SLO/SLA dashboards, incident review
- **Security & DevSecOps:** IAM least privilege, secrets management, vulnerability scanning, container scanning, SAST, branch protection, audit logging, encryption
- **Scripting & Automation:** **Bash**, Python, PowerShell, YAML, JSON, Makefile, CLI automation, release scripts, infrastructure validation
- **Databases & Runtime Support:** PostgreSQL, MySQL, Redis, MongoDB, backup automation, restore testing, migration support, performance troubleshooting

Experience

Lightedge — Senior Software Engineer (Apr 2020 – Mar 2026)

- Led **AWS** infrastructure modernization for SaaS and managed-service platforms using **Terraform 0.12–1.8**, **EKS Kubernetes 1.18–1.29**, **Docker 19.x–26.x**, and **GitHub Actions** across production, staging, and disaster-recovery environments.
- Designed reusable **Terraform modules** for VPC, IAM, ALB, RDS, S3, CloudFront, Route 53, and EKS node groups, reducing new environment provisioning time while keeping naming, tagging, and policy rules consistent.
- Built secure **CI/CD pipelines** with **GitHub Actions**, **Jenkins 2.x**, container scanning, branch protection, approval gates, and blue-green release patterns that reduced failed production deployments.
- Migrated legacy EC2-based deployments into **Docker** and **Kubernetes** workloads with Helm charts, readiness probes, resource limits, autoscaling policies, and rollback plans that improved release reliability without stopping client traffic.

- Resolved a recurring production outage caused by unhealthy pods passing incomplete checks by adding deeper **readiness probes**, ALB target validation, graceful shutdown hooks, and automated rollback conditions.
- Implemented centralized **observability** with **Prometheus 2.x**, **Grafana 7.x–10.x**, CloudWatch metrics, structured logs, and alert routing, cutting average incident detection time by **45%**.
- Hardened cloud access by replacing broad IAM permissions with least-privilege roles, service-account boundaries, encrypted secrets, SSO groups, and audit-ready policies across multi-account AWS environments.
- Created a developer self-service workflow using reusable pipeline templates, container build standards, and environment promotion rules, helping engineers deploy safely without waiting for manual DevOps support.
- Fixed a costly scaling issue in EKS where over-provisioned worker nodes stayed idle by tuning HPA, Cluster Autoscaler, requests, limits, and node group sizing to reduce monthly compute spend by **29%**.
- Introduced **Argo CD 1.x–2.x** GitOps deployment patterns with Helm values per environment, drift detection, pull-request reviews, and controlled sync windows for safer configuration changes.
- Automated database backup validation, restore drills, snapshot retention, and encrypted artifact storage to improve recovery confidence for customer-facing services handling sensitive operational data.
- Partnered with security, backend, and product teams to balance speed and governance by using **Terraform**, **Kubernetes**, **CI/CD**, **monitoring**, and incident reviews flexibly across infrastructure and application needs.

NIX United — Software Engineer *(Oct 2016 – Mar 2020)*

- Built DevOps foundations for eCommerce and healthcare applications using **AWS EC2**, **ALB**, **RDS**, **S3**, **Docker 1.12–19.x**, **Jenkins 2.x**, **GitLab CI 8.x–12.x**, and **Ansible 2.4–2.9**.
- Created automated build and deployment jobs for PHP, Node.js, and frontend services, replacing manual SSH releases and reducing average deployment effort by **52%** across client environments.
- Migrated older shell-script deployments into **Jenkins 2.x** pipelines with artifact versioning, environment variables, rollback steps, and approval stages to make releases more traceable and repeatable.
- Implemented **Ansible playbooks** for Linux hardening, package installation, Nginx configuration, Supervisor services, cron jobs, and application bootstrapping across staging and production servers.
- Resolved a checkout downtime issue caused by inconsistent Nginx and PHP-FPM restart order by adding health checks, service dependencies, and automated post-deployment verification.
- Introduced Dockerized local development for backend services, Redis, MySQL, and queue workers, reducing setup errors for new engineers by **41%** and making environment behavior more predictable.
- Configured CloudWatch alarms, log rotation, disk alerts, and application-level uptime checks so support teams could identify memory leaks, failed workers, and slow API responses earlier.
- Improved release safety by adding branch-based deployment rules, database migration checks, build artifact retention, and rollback commands for high-traffic eCommerce release windows.
- Fixed intermittent CI failures caused by dependency cache corruption by isolating build workspaces, pinning package versions, and rebuilding cache layers only after verified lockfile changes.
- Supported early **Kubernetes 1.8–1.17** adoption for selected services by preparing Docker images, Helm templates, service manifests, and resource boundaries while keeping stable workloads on EC2.
- Worked flexibly across infrastructure, backend troubleshooting, and release engineering, using **Linux**, **Bash**, **AWS**, **Docker**, and **CI/CD** to unblock teams during urgent delivery cycles.

Webiz Digital — Software Developer *(Jan 2014 – Sep 2016)*

- Supported web platform operations for digital commerce and marketing systems using **Linux**, **Bash**, **Jenkins 1.x–2.x**, **Nginx**, **Apache**, **MySQL**, **Ansible 1.9–2.2**, and Git-based release workflows.
- Automated repetitive server setup tasks with Bash scripts and early **Ansible** roles, reducing manual configuration mistakes across shared staging and production environments.

- Maintained Jenkins build jobs for PHP and JavaScript applications, adding lint checks, test commands, packaged artifacts, and deployment logs so release failures became easier to diagnose.
- Resolved a production issue where static assets were served from stale directories by improving Nginx cache rules, deployment symlinks, and post-release validation scripts.
- Implemented log rotation, disk-space monitoring, backup scripts, and MySQL dump verification to prevent common outages on smaller client-hosted infrastructure.
- Helped migrate manual FTP-style deployments into Git-based release packages with tagged versions, SSH deployment scripts, and rollback folders that improved release consistency.
- Introduced basic Docker 1.x usage for isolated development experiments while keeping production stable on conventional Linux, Nginx, Apache, and MySQL hosting patterns used widely at that time.
- Fixed intermittent environment drift between developers and staging by documenting OS packages, PHP extensions, Node versions, cron behavior, and required environment variables.
- Created simple operational dashboards from server logs, uptime checks, and deployment notes, helping account managers explain incidents and reducing repeated support questions by **27%**.
- Collaborated with developers and designers to use **Linux, Bash, Jenkins**, and web-server configuration flexibly whenever delivery required both application and infrastructure support.

Aurio — Junior Software Developer *(Oct 2012 – Dec 2013)*

- Supported early-stage infrastructure operations for web applications using **Linux, Bash, Jenkins 1.x, Apache 2.2–2.4, Nginx 1.x, MySQL 5.x**, SVN, Git, and cron-based automation.
- Maintained staging and production server checklists for deployments, permissions, log locations, service restarts, and rollback steps to reduce avoidable release mistakes.
- Resolved recurring website downtime caused by full log directories by adding logrotate rules, disk usage alerts, and cleanup scripts for Apache, Nginx, and application logs.
- Built Bash scripts for backup rotation, file synchronization, process checks, and deployment preparation, reducing routine operational work by **31%** for small web projects.
- Assisted migration from SVN-heavy workflows toward Git-based branching, tagged release archives, and clearer change history so developers could coordinate fixes more safely.
- Configured basic monitoring with Nagios-style checks, uptime alerts, and manual incident notes to help the team understand server health before customer complaints arrived.
- Improved MySQL backup reliability by validating dump output, checking restore commands on staging, and separating configuration files from generated backup artifacts.
- Fixed permission-related deployment errors by standardizing Linux user groups, directory ownership, web-server write paths, and secure SSH access patterns across environments.
- Worked across junior development, release support, and server maintenance tasks, using **Linux, Bash, web servers**, and troubleshooting discipline to grow into DevOps-focused engineering.

Microsoft — Software Developer Intern *(Jun 2012 – Sep 2012)*

- Supported internal engineering tools using **C#, .NET Framework 4.0–4.5, ASP.NET MVC 3–4**, JavaScript, jQuery, and SQL Server, gaining practical experience in enterprise software quality and structured development workflows.
- Assisted senior engineers with debugging UI defects, validation issues, and data-handling problems, documenting root causes clearly so fixes could be reviewed and released with lower regression risk.
- Built small internal web modules for tracking records, reviewing status changes, and improving team visibility, applying clean HTML, CSS, JavaScript, and backend integration patterns.
- Resolved early-stage bugs related to inconsistent form validation and SQL query results by reproducing issues locally, checking logs, and working with mentors to apply safer input-handling rules.

Education

Texas State University
Bachelor's Degree in Computer Science (*Sep 2008 – May 2012*)